Campus Lost & Found — Use Case Specifications

The workflows drawn as activity diagrams only show the *shape* of each process. This document pins down the exact wording, statuses, and actor names so that whoever draws the use case diagram, whoever draws the SSDs, and whoever draws the class diagram all end up describing the **same system** — even if you're working in parallel.

Shared Vocabulary (use these exact terms on every diagram)

Actors — use these names everywhere, don't invent synonyms:

- `Finder` — a user reporting an item they found.
- `Owner` — a user reporting/searching for an item they lost.
- `Admin – reserved for future moderation. Not implemented in v1: the User.role field exists, but there are no admin-only screens or endpoints, so don't draw admin use cases as part of the current system.`
- `System` — the application itself (used as the lifeline/participant in SSDs).

Note: `Finder` and `Owner` are *roles*, not separate classes — the same `User` can be both at different times. Don't create a `Finder` or `Owner` class in the domain model.

Status values — use exactly these strings, not "active"/"pending"/etc. interchangeably:

- `LostItemPost.status: OPEN → RECOVERED (the owner marks it recovered, or it is auto-recovered when the owner's claim on a matching found item is approved – §5). FoundItemPost.status: OPEN → PENDING_CLAIM (a claim was submitted) → CLAIMED (claim approved) or back to OPEN (claim rejected).`
- `Claim.status: PENDING → APPROVED or REJECTED.`
- `Notification.isRead: false (unread) / true (read).`

Method/message names — use these on SSDs so arrows match across diagrams:

- `reportFoundItem(details)`
- `reportLostItem(details)`

- `searchItems(criteria)`
- `viewItemDetails(postId)`
- `submitClaim(foundItemPostId, justification, itemDetails, proofPhotoUrl)`
- `reviewClaim(claimId, decision)`
- `markRecovered(postId)`
- `deletePost(postId) – the post's owner deletes their own post (hard delete)`
- `runMatching(newPostId) – internal, System-only; runs automatically right after a report succeeds`
- `notifyUser(userId, message, relatedItemId, relatedClaimId?) – internal, System-only`

- `sendClaimInvite(foundItemPostId, lostItemId) – "I have this": a Finder invites a lost post's owner to claim one of their found posts`

Whoever draws the domain class diagram should treat this list as the method signatures for the relevant service classes (e.g. `ItemPostService.reportFoundItem(...)`), so the SSD arrows and the class diagram's operations line up.

1. Report Found Item

Actor: Finder. **Precondition:** Finder is logged in. **Trigger:** Finder physically found an item and wants to log it in the system.

Finder fills out a form with the item's title, category, description, an optional photo, where it was found, and when. Photos upload through the media service (Cloudinary) before the post is created. On submit, the System validates the input (all diagrams should show a validation-failure branch — required fields, image type and size). If valid, the System creates a new FoundItemPost with status OPEN, then immediately runs the matching check (see §6) against existing open lost-item posts.

Postcondition: A new FoundItemPost exists with status OPEN. Zero or more Notifications may have been created as a side effect, but that is not part of this use case's main success scenario — treat matching as a separate, triggered use case so the activity diagram for this one doesn't get cluttered.

2. Report Lost Item

Actor: Owner. **Precondition:** Owner is logged in. **Trigger:** Owner realizes they lost an item and wants to log it.

Mirror image of §1: Owner submits title, category, description, last-seen location, and date. Same validation branch. On success, System creates a LostItemPost with status OPEN, then triggers matching (§6) against existing open found-item posts.

Postcondition: same shape as §1, just for `LostItemPost`.

3. Search / Browse Items

Actor: any logged-in User (Finder or Owner role doesn't matter here — anyone can search). **Precondition:** none beyond being logged in (assumption — see note below).

User enters any combination of keyword, category, location, or status and submits — the directory pages default to OPEN but can filter any status, and the header's global search box queries across both post types at once. System returns a paginated list of matching posts (both found and lost). If nothing matches, show an empty-state message rather than an error — this isn't a failure case, just a normal outcome, so don't draw it as an exception branch.

Note for the team: we're assuming browsing requires login for simplicity (so we don't have to design a public/guest view). If anyone wants guest browsing later, that's a scope change worth flagging, not something to quietly draw differently on one diagram.

Postcondition: none — this is a read-only use case, nothing is created or changed.

4. View Item Details

Actor: any logged-in User. Precondition: the post exists. Any status is viewable — claimed and recovered posts stay browsable (the directories just default to the OPEN filter), so owners and claimants can revisit history.

Straightforward: User selects a post from search results, System returns full details (all fields plus photo and the post owner's profile). This is usually drawn as the tail end of the Search workflow (§3) rather than as its own separate activity diagram — treat it as a continuation, not a standalone flow, so we don't end up with two diagrams both claiming to "start" at search results.

5. Claim Found Item

Actor: Owner (primary), Finder (secondary — receives and reviews the claim). Precondition: a FoundItemPost exists with status OPEN, it is not the Owner's own post, and the Owner has no other pending claim on it.

Owner views a found post they believe is theirs and clicks "This is mine," filling in a justification, item details that only the real owner would know, and an optional proof photo. System creates a Claim with status PENDING, sets the found post to PENDING_CLAIM, and notifies the Finder with a deep link to the claim's detail page. Finder reviews the claim details and either approves or rejects it.

- **If approved: Claim.status → APPROVED and FoundItemPost.status → CLAIMED. In addition, the Owner's open lost-item posts that the System links to this found item — by the matching rule (§6) or by a claim invite (§8) — are automatically set to RECOVERED, so both sides of the story close together. The two users are expected to arrange the physical handover themselves — the system does not manage the handover logistics (that's out of scope, don't draw a "meet up" step as a system-tracked activity).**
- **If rejected: Claim.status → REJECTED, and FoundItemPost.status returns to OPEN (someone else can still claim it).**

Important for whoever draws this activity diagram: there are two actors doing different things at different times (Owner claims, Finder reviews) — this is the best candidate for a proper two-swimlane diagram (Owner | System | Finder, or just User-side | System if you want to keep it to two lanes like the textbook example).

6. Receive Match Notification

Actor: System (this use case has no *human* initiator — it's triggered by §1 or §2, not by a person clicking anything). Secondary actor: the User who ends up receiving the notification.

Whenever a new post is created, System runs a matching check: it compares the new post against open posts of the opposite type using a simple score — same category plus keyword overlap; candidates scoring at least 0.6 count as matches (v1's rule — a smarter Strategy can plug in later). Both sides are notified: the lost-item owner gets a notification pointing at the found post (so they can review and claim it), and the finder gets one pointing at the lost post (so they can send a claim invite, §8). Clicking a notification navigates to the related item's page.

Why this matters for your diagrams: don't draw this as something a user "does" — on the use case diagram it should connect to `System` as the actor, not to Finder/Owner. On the activity diagram, there's no user swimlane at the start; the user only appears at the very end when they open the notification (that's use case §4, reused).

7. Mark Lost Item as Recovered

Actor: Owner. Precondition: a LostItemPost exists, owned by this Owner, status OPEN. Owner. **Precondition:** a `LostItemPost` exists, owned by this Owner, status `open`.

Owner opens their own lost-item post and marks it recovered — this covers the case where they found the item themselves, or got it back informally without going through the Claim flow. System asks for confirmation, then sets status → RECOVERED. No Claim object is involved here; claims only exist for found-item posts.

Don't confuse this with §5: §5 is initiated on a *found* post by an Owner claiming it; §7 is initiated on the Owner's *own lost* post, with no Finder involved at all.

8. Delete Own Post

Actor: Owner (the post's owner). Precondition: post exists and belongs to this user. Admin moderation was dropped for v1 — there are no admin screens or endpoints — so removal is self-service.

From My Posts (or the item's detail dialog), the owner confirms deletion and the System hard-deletes the post. Closing a post the "record-keeping" way happens through the claim (§5) and recover (§7) flows instead; deletion is the only remove path.

9. Claim Invite — "I Have This"

Actor: Finder (primary), Owner (secondary — receives the invite). Precondition: a LostItemPost with status OPEN exists, the Finder is not its owner, and the Finder has at least one of their own FoundItemPost posts with status OPEN to point at.

Viewing someone else's lost post, the Finder clicks "I have this" and picks which of their open found posts is the match. The System records a ClaimInvite (the found-lost pair; sending the same pair twice is rejected) and notifies the Owner with a deep link to that found post, where they submit a normal claim (§5).

Why it exists: it is the manual escape hatch for pairs the matching rule (§6) would never link — a human asserts "these two are the same item." When that claim is approved, the invited lost post is auto-recovered even without an algorithmic match. Postcondition: a ClaimInvite exists linking the two posts; everything after that is the §5 flow.

10. Manage Profile

Actor: any logged-in User. The profile screen updates name, email, password, "about," and avatar (image upload). Pure account management — it feeds nothing in the matching or claim flows. The session lives in an HTTP-only cookie; the frontend stores only the displayed user object.

11. Register / Login / Password Reset

Actor: Visitor. Register creates an account (name, email, password) and Login issues a session cookie — "remember me" extends it from 1 day to 30 days. Forgot password emails a signed link that expires in 15 minutes; completing the reset changes the password and confirms by email. These are the entry points for every other use case.
